

LUXXFOCUSS

Developer Hiring Test — Official Candidate Assignment

Position	Developer
Assessment Type	Practical Technical Assessment
Estimated Duration	48–72 Hours
Project	Luxxfocuss Website & Digital Platform

1. Purpose of the Assessment

This technical assessment is designed to evaluate your ability to work on a real-world production website and demonstrate your skills in:

- Frontend development
- Backend/API development
- Database design
- UI/UX implementation
- Responsive development
- Performance optimization
- Debugging
- Security
- Code architecture
- Problem-solving
- Product thinking

You are expected to approach this assignment as if you have joined the **Luxxfocuss development team** and have been asked to improve and extend the existing platform.

2. Existing Website

Please review the existing Luxxfocuss website before beginning the task:

Website: <https://luxxfocuss.vercel.app/>

Your first responsibility is to understand the existing website, its user flow, design language, functionality, and technical implementation.

Do not immediately start coding. First understand the product.

3. TASK 01 — Website Audit

Perform a technical and UX audit of the existing Luxxfocuss website.

Identify:

UI/UX

- Navigation issues

- Layout problems
- Typography issues
- Spacing inconsistencies
- CTA placement
- User-flow problems
- Mobile usability issues
- Visual consistency

Technical

- Console errors
- Broken links
- Broken components
- Unnecessary API requests
- Poor component structure
- Duplicate code
- Potential performance issues
- Potential security issues

Performance

Review:

- Page loading
- Image optimization
- JavaScript bundle usage
- Lazy loading
- Caching opportunities
- Core Web Vitals
- Mobile performance

SEO

Review:

- Page titles
- Meta descriptions
- Heading hierarchy
- Image alt attributes
- Open Graph metadata
- Sitemap
- Robots configuration
- Semantic HTML

Deliverable

Create `AUDIT.md` or `AUDIT.pdf`. The report should clearly explain:

1. Problem
2. Why it matters
3. Recommended solution

4. Priority

Use the following priority system:

Critical → High → Medium → Low

4. TASK 02 — Improve the Existing Website

Select the most important issues identified during your audit and implement meaningful improvements. You should improve the existing design rather than completely replacing the Luxxfocuss branding.

Focus on:

- Better responsive behavior
- Cleaner UI
- Better spacing
- Better typography
- Improved CTA experience
- Better navigation
- Better component structure
- Improved loading experience
- Better accessibility

You should be able to explain **why** each major change was made.

5. TASK 03 — Responsive Development

The website must work properly across:

Device	Viewport widths
Desktop	1920px, 1440px, 1280px
Tablet	1024px, 768px
Mobile	430px, 390px, 375px

Test the following:

- Navbar
- Hero sections
- Cards
- Images
- Buttons
- Forms
- Sections
- Footer
- Typography
- Spacing

There should be:

- No horizontal scrolling

- No overlapping elements
- No broken images
- No unreadable text
- No unusable buttons

6. TASK 04 — Build One Real Functional Feature

Implement one meaningful feature for the Luxxfocuss platform.

Required Example: Contact / Inquiry Management

Create a complete inquiry system.

Frontend

The user should be able to submit:

- Name
- Email
- Phone
- Message

Include:

- Form validation
- Loading state
- Success state
- Error state
- Proper validation messages

Backend

Create an API that handles the inquiry. Example:

```
POST    /api/contact
GET     /api/contact
GET     /api/contact/:id
PATCH  /api/contact/:id
DELETE  /api/contact/:id
```

You may modify the API structure if your architecture requires it.

7. TASK 05 — Admin Management

Create a simple admin interface where inquiries can be viewed and managed. The admin should be able to see:

```
Name
Email
Phone
Message
Created Date
Status
```

Status options:

```
New
Contacted
In Progress
Completed
Cancelled
```

The admin should be able to change the status.

8. TASK 06 — Database

Create an appropriate database structure. For example:

```
users
contacts
```

A contact record could contain:

```
id
name
email
phone
message
status
created_at
updated_at
```

Use appropriate:

- Data types
- Primary keys
- Relationships
- Constraints
- Indexes

Explain your database decisions in the README.

9. TASK 07 — Authentication & Security

If an admin dashboard is implemented, it must not be publicly accessible. Implement appropriate authentication. Depending on your technology stack, you may use:

- JWT
- Session authentication
- Secure cookie authentication
- OAuth

At minimum, demonstrate:

- Protected admin routes
- Authentication validation
- Input validation
- Proper error handling
- Protection against unauthorized requests

Do not expose sensitive credentials in the repository.

10. TASK 08 — Performance Optimization

Measure the website performance before and after your changes. Use an appropriate performance testing tool such as Lighthouse. Document:

	Before	After
Performance		
Accessibility		
Best Practices		
SEO		

Explain what you changed to achieve the improvement. Possible areas include:

- Image optimization
- Lazy loading
- Code splitting
- Removing unnecessary dependencies
- Reducing API calls
- Caching
- Optimized rendering

11. TASK 09 — Error Handling

The application should gracefully handle common errors. Examples:

```
API unavailable
Invalid form input
Database error
Unauthorized request
404 page
Empty data
Slow network
Server error
```

Do not leave users with blank screens or raw technical errors.

12. TASK 10 — Code Quality

Your code should demonstrate professional development practices. Evaluate your own implementation for:

- Clean architecture
- Reusable components
- Meaningful naming
- Separation of concerns

- Avoidance of unnecessary duplication
- Maintainability
- Type safety where applicable
- Consistent formatting
- Proper error handling

Do not submit unnecessary or unused code.

13. Technology Stack

You may use technologies appropriate for the project. Examples include:

Frontend

- React
- Next.js
- Vue
- TypeScript
- Tailwind CSS
- CSS Modules

Backend

- Node.js
- Next.js API
- Express
- NestJS
- Other suitable backend technology

Database

- PostgreSQL
- MySQL
- MongoDB
- Supabase
- Firebase

You must explain your technology choices.

Do not choose a technology simply because it is popular. Choose what you can implement properly and maintain.

14. Git & Version Control

Use Git throughout the project. Your repository should contain meaningful commits. Examples:

```
Initial project setup
Improve responsive navigation
Implement contact form
Add contact API
Add database integration
Implement admin dashboard
Improve performance
Fix mobile layout
```

Avoid submitting the entire project with a single commit such as:

```
final
final2
final-final
done
```

15. Documentation

Your repository must contain `README.md`. Include:

Project Overview

Explain what you built.

Technology Stack

List the technologies used.

Installation

Explain how to run the project locally.

Environment Variables

Explain required environment variables without exposing real secrets. Example:

```
DATABASE_URL=
JWT_SECRET=
NEXT_PUBLIC_API_URL=
```

Database Setup

Explain how to initialize the database.

API Documentation

Document the APIs you created.

Architecture

Briefly explain your project structure.

Testing

Explain how you tested the application.

Known Issues

Clearly mention anything unfinished.

Future Improvements

Explain what you would build next.

16. Deployment

Provide a working live deployment. Submit:

Live URL: _____

GitHub Repository: _____

The evaluator should be able to access the deployed application and test the implemented features.

17. Demo Video

Record a short **3-5 minute demo video**. The video should demonstrate:

1. Existing website understanding
2. Major improvements
3. New feature
4. User flow
5. Admin functionality
6. Responsive behavior
7. Performance improvements

You do not need a highly edited video. A clear screen recording is sufficient.

18. Product Thinking Challenge

Imagine you are joining Luxxfocuss as a full-time developer. Answer the following:

“What would you improve in Luxxfocuss over the next 3 months?”

Create a short roadmap.

Month 1 — Foundation

- Bug fixing
- UI/UX improvements
- Responsive improvements
- Performance
- Code cleanup

Month 2 — Platform

- Backend improvements
- Admin system
- Authentication
- Analytics
- Automation

Month 3 — Growth & Scalability

- SEO
- Conversion improvements
- Advanced analytics
- Security
- Scalability
- New product features

Explain **why** you selected each improvement.

19. Submission Requirements

Submit all of the following:

Required	Optional
GitHub repository	Architecture diagram
Live deployment	Figma/UI concepts
README.md	Automated tests
Website audit	CI/CD configuration
Database schema	Additional improvements
API documentation	
Performance report	
Demo video	

20. Evaluation Criteria

The assessment will be evaluated out of **100 points**.

Category	Points
Understanding of Existing Website	15
Code Quality & Architecture	15
UI/UX Implementation	15
Functional Development	15
Backend & API	10
Responsive Design	10
Performance Optimization	10
Security & Error Handling	5
Documentation	5
Total	100

21. What We Are Looking For

We are not only evaluating whether the application works. We want to understand how you think as a developer. We will look for:

- Problem-solving ability
- Ability to understand an existing codebase
- Clean and maintainable code
- Attention to detail
- Understanding of user experience
- Ability to work independently
- Technical decision-making
- Debugging ability
- Performance awareness
- Security awareness
- Product thinking

22. Important Instructions

1. Do not copy an existing template and submit it as your own work.
2. You may use documentation, libraries and development tools.
3. If you use AI-assisted development tools, you are expected to understand and be able to explain the code you submit.
4. Do not expose API keys, passwords, database credentials or other secrets.
5. If you cannot complete a feature, document what remains unfinished and explain how you would complete it.
6. Quality is more important than adding a large number of unnecessary features.
7. You should prioritize real problems over cosmetic changes.

23. Final Candidate Checklist

Before submitting, verify:

```
[ ] Website reviewed
[ ] Technical audit completed
[ ] UI/UX improvements completed
[ ] Mobile responsive
[ ] Functional feature implemented
[ ] API implemented
[ ] Database connected
[ ] Admin functionality implemented
[ ] Authentication implemented where required
[ ] Validation implemented
[ ] Error handling implemented
[ ] Performance tested
[ ] GitHub repository ready
[ ] README completed
[ ] Live deployment working
[ ] Demo video recorded
[ ] No secrets exposed
```

24. Final Submission

Please submit:

Candidate Name	
Position	
GitHub	
Live Demo	
Demo Video	
Audit Report	
Expected Completion Date	

Luxxfocuss Development Team

Assessment Principle:

*Build with purpose.
Solve real problems.
Write maintainable code.
Think like a product engineer.*

Good luck.